OPERATING SYSTEMS

UNIT III – PROCESS SYNCHRONIZATION & DEADLOCKS

SECTION A – 2-MARK QUESTIONS (3-MARK ANSWER FORMAT)

Answers, examples and simple diagrams

Q1. Define Busy Waiting.

Definition:

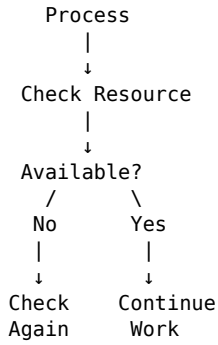
Busy Waiting is a method in which a process continuously checks whether a resource is available or not.

Explanation:

- The process keeps checking again and again.
- It does not do other work while waiting.
- It wastes CPU time.

Example: A person repeatedly checking whether a door is open or not.

Diagram:



Q2. What is Starvation?

Definition:

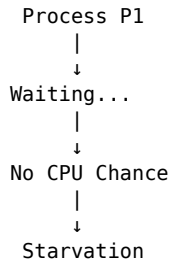
Starvation is a condition in which a process waits for a long time because it does not get the required CPU or resources.

Explanation:

- Some processes get priority.
- Low-priority processes may wait continuously.
- The waiting process may never get a chance.

Example: A student waits for a chance to speak, but the teacher always selects other students.

Diagram:



Q3. State Peterson's Solution for the Critical Section Problem.

Definition:

Peterson's Solution is a software method used to solve the critical section problem between two processes.

Explanation:

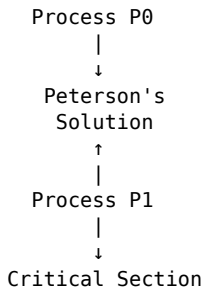
- It uses two variables: flag[] and turn.
- flag[] tells whether a process wants to enter the critical section.
- turn decides which process gets a chance.

It provides:

Mutual Exclusion Progress Bounded Waiting

Example: Two students want to use one computer. Peterson's solution decides who can use it first.

Diagram:



Q4. Define Deadlock in Operating Systems.

Definition:

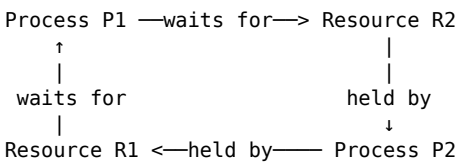
Deadlock is a situation where two or more processes wait for each other and none of them can continue.

Explanation:

- Each process holds a resource.
- It waits for another resource held by another process.
- No process can continue its work.

Example: Two persons are holding one pencil each and waiting to exchange them. Neither person gives their pencil first.

Diagram:



Q5. What is a Semaphore? / Define Semaphores.

Definition:

A Semaphore is a special variable used by the Operating System to control access to shared resources.

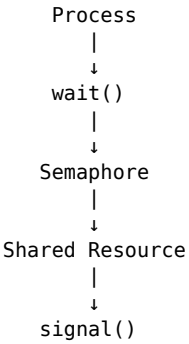
Explanation:

- It is used for process synchronization.
- It uses two operations: wait() and signal().

- It prevents multiple processes from accessing a shared resource incorrectly.

Example: A parking area has only one available space. A semaphore controls who can use it.

Diagram:



Q6. Mention two conditions necessary for deadlock to occur.

Definition:

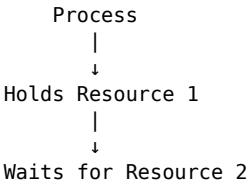
Deadlock occurs when certain conditions are present in a computer system.

Explanation:

- Mutual Exclusion: Only one process can use a resource at a time.
- Hold and Wait: A process holds one resource and waits for another resource.

Example: One person holds a pen and waits for a book, while another person holds the book.

Diagram:



Q7. Define Race Condition.

Definition:

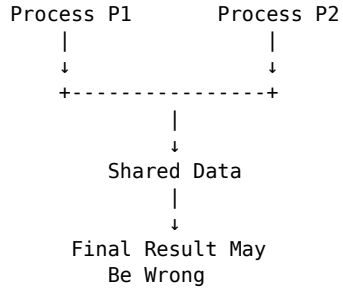
A Race Condition occurs when two or more processes access shared data at the same time, and the final result depends on the order of execution.

Explanation:

- Multiple processes use the same data.
- They try to change the data at the same time.
- This may produce an incorrect result.

Example: Two persons try to withdraw money from the same bank account at the same time.

Diagram:



Q8. What are the requirements that a solution to the Critical Section Problem must satisfy?

Definition:

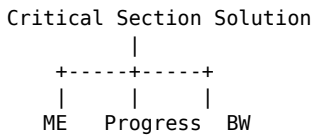
A Critical Section Problem solution must satisfy three important requirements to allow processes to share resources safely.

Explanation:

- Mutual Exclusion: Only one process can enter the critical section at a time.
- Progress: If no process is inside the critical section, a waiting process must get a chance without unnecessary delay.
- Bounded Waiting: Every process must get a chance within a limited waiting time.

Example: Two students want to use one computer. Only one student can use it at a time, and both must get a fair chance.

Diagram:



Q9. Name the classic problems of Synchronization.

Definition:

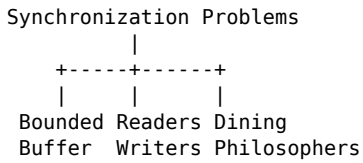
Classic Synchronization Problems are common problems used to understand how processes share resources safely.

Explanation:

- Bounded Buffer Problem: Producers and consumers share a limited-size buffer.
- Readers-Writers Problem: Multiple readers and writers access shared data.
- Dining Philosophers Problem: Philosophers share limited forks to eat food.

Example: Many students sharing a limited number of computers in a lab.

Diagram:



Q10. What is the Critical Section Problem?

Definition:

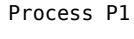
The Critical Section Problem occurs when multiple processes try to access shared data or resources at the same time.

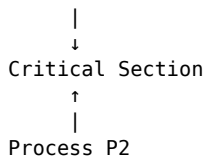
Explanation:

- Critical Section is the part of a program where shared data is accessed.
- Only one process should enter the critical section at a time.
- It prevents data errors and race conditions.

Example: Two people trying to update the same bank account balance.

Diagram:





Q11. Define Spinlock.

Definition:

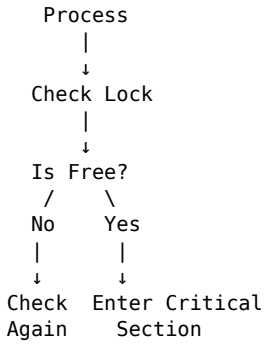
A Spinlock is a type of lock in which a process continuously checks whether the lock is available.

Explanation:

- A process tries to acquire a lock.
- If the lock is busy, it keeps checking.
- When the lock becomes free, the process enters the critical section.

Example: A person continuously checking whether a computer is available.

Diagram:



Q12. What is Process Synchronization?

Definition:

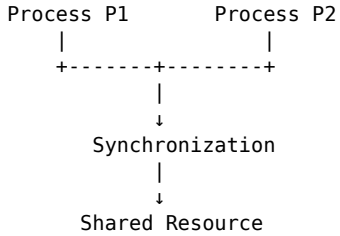
Process Synchronization is a method used to coordinate multiple processes when they share data or resources.

Explanation:

- It controls the execution of processes.
- It prevents conflicts when sharing resources.
- It maintains correct results.

Example: Two students sharing one computer must use it one after another.

Diagram:



Q13. List the four necessary conditions for a Deadlock to occur.

Definition:

Four necessary conditions must be present together for a deadlock to occur.

Explanation:

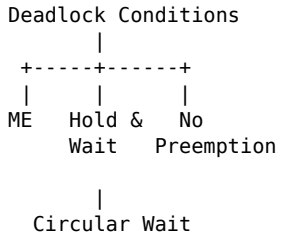
- Mutual Exclusion: Only one process can use a resource at a time.
- Hold and Wait: A process holds one resource and waits for another.
- No Preemption: A resource cannot be taken forcefully from a process.
- Circular Wait: Processes wait for each other in a circular order.

Remember:

All four conditions must exist together for a deadlock to occur.

Example: Two people hold different books and wait for each other's books.

Diagram:



Q14. Differentiate between Deadlock Prevention and Deadlock Avoidance.

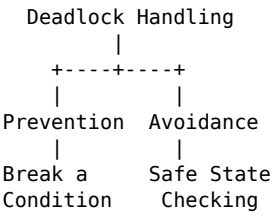
Definition:

Deadlock Prevention and Deadlock Avoidance are two methods used to handle deadlocks.

Differences:

Deadlock Prevention: Prevents at least one necessary deadlock condition; stops deadlock before it can occur; uses restrictions on resource usage; Example: Breaking circular wait. Deadlock Avoidance: Checks whether resource allocation is safe; avoids unsafe resource allocation; uses information about future resource needs; Example: Banker's Algorithm.

Diagram:



Q15. What is a Wait-for Graph and Resource-Allocation Graph?

Definition:

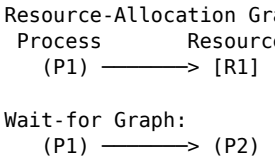
A Resource-Allocation Graph (RAG) is a diagram used to show the relationship between processes and resources. A Wait-for Graph (WFG) shows which process is waiting for another process.

Explanation:

- In RAG, circles represent processes and boxes represent resources.
- A process-to-resource arrow means the process is requesting a resource.
- A resource-to-process arrow means the resource is allocated to that process.

Example: Process P1 is waiting for a resource held by P2.

Diagram:



```
Waiting for
P2
```

Q16. What is a Safe State in Deadlock Avoidance?

Definition:

A Safe State is a condition in which the system can allocate resources to all processes in some order without causing a deadlock.

Explanation:

- All processes can complete their work.
- The system has a safe sequence.
- Deadlock can be avoided by following that sequence.

Example: Three students need computers. The available computers can be given in an order that allows all three students to finish their work.

Diagram:

```
  Safe State
      |
      ↓
Safe Sequence
      |
      ↓
  Process P1
      |
      ↓
  Process P2
      |
      ↓
  Process P3
      |
      ↓
All Completed
```

Unit III – Quick Revision

Q.No	Topic	Important Keywords
1	Busy Waiting	Continuous Checking, CPU Time
2	Starvation	Long Waiting, No Chance
3	Peterson's Solution	Flag, Turn
4	Deadlock	Processes, Waiting
5	Semaphore	Wait, Signal
6	Deadlock Conditions	Mutual Exclusion, Hold and Wait
7	Race Condition	Shared Data, Incorrect Result
8	Critical Section Requirements	Mutual Exclusion, Progress, Bounded Waiting
9	Synchronization Problems	Bounded Buffer, Readers-Writers, Dining Philosophers
10	Critical Section	Shared Resource
11	Spinlock	Lock, Continuous Checking
12	Process Synchronization	Coordination, Shared Resources

Q.No	Topic	Important Keywords
13	Four Deadlock Conditions	ME, Hold & Wait, No Preemption, Circular Wait
14	Prevention vs Avoidance	Break Condition, Safe State
15	Wait-for Graph	Processes, Waiting
16	Safe State	Safe Sequence